



ÉCOLE CENTRALE CASABLANCA

RAPPORT

Autoencodeurs pour le clustering et la détection d'anomalies : application en Maintenance Prédictive

Élèves :

Vincent MENDY

Rosteim Falleiz BELEMCOABGA

Abdoul-Nasser BARRY

Pegdwendé Anita Sylviane

OUEDRAOGO

Encadrants :

Youssef LAMRANI

Table des matières

1	Introduction	2
2	Fondements Théoriques	2
2.1	Détection d'anomalies non supervisée	2
2.2	Les autoencodeurs : principe de base	3
2.3	Méthodes classiques de référence	6
2.4	Clustering non supervisé dans l'espace latent	8
2.4.1	K-Means	9
2.4.2	DBSCAN	9
2.5	Visualisation de l'espace latent	9
3	Exploration et préparation des données (EDA)	9
4	Modélisation avec Autoencodeurs	10
4.1	Implémentation Autoencodeur Dense	10
4.2	Implémentation de l' Autoencodeur LSTM	12
5	Comparaison avec les méthodes classiques	13
5.0.1	Interprétation des résultats des modèles de détection de pannes . . .	17
6	Clustering dans l'espace latent	17
7	Visualisation et interprétation	18
8	Développement du tableau de bord Streamlit	20
9	Conclusion	22

1 Introduction

La maintenance prédictive est une stratégie clé dans l'industrie moderne, visant à anticiper les pannes des équipements afin de réduire les coûts et les temps d'arrêt. Elle s'appuie sur l'analyse de données issues de capteurs pour détecter des signes précurseurs de défaillances. Dans ce contexte, les méthodes d'apprentissage automatique non supervisé, en particulier les autoencodeurs, offrent une solution puissante pour détecter les anomalies sans nécessiter un grand nombre d'exemples étiquetés de pannes.

Ce projet, basé sur le jeu de données synthétique AI4I 2020 Predictive Maintenance Dataset, met en œuvre et compare plusieurs approches de détection d'anomalies : autoencodeurs denses et LSTM comme méthodes principales, ainsi que des algorithmes classiques (Isolation Forest, One-Class SVM, Local Outlier Factor) comme références. Nous intégrons également du clustering dans l'espace latent et développons un tableau de bord interactif avec Streamlit.

Les objectifs respectent les consignes : présentation théorique détaillée, analyse exploratoire des données (EDA), modélisation, comparaison, clustering, visualisation et déploiement. Ce rapport est structuré pour expliquer en profondeur les concepts théoriques, avec des illustrations et des exemples pour favoriser une compréhension académique et technique.

2 Fondements Théoriques

Cette section présente les concepts fondamentaux utilisés dans le projet. Chaque méthode est expliquée de manière progressive : principe de base, mécanisme interne, pourquoi elle est adaptée à la maintenance prédictive, et ses limites principales. Des illustrations et des équations mathématiques sont incluses pour clarifier les explications.

2.1 Détection d'anomalies non supervisée

En maintenance prédictive, les données d'anomalies (pannes réelles) sont très rares par rapport aux données normales, représentant souvent moins de 5 % des échantillons. L'apprentissage supervisé classique, qui nécessite des labels équilibrés, devient donc difficile voire impossible en raison du déséquilibre des classes.

Principe général : On entraîne un modèle exclusivement sur des données considérées comme normales (comportements typiques des machines). Le modèle apprend à modéliser cette « normalité ». Tout ce qui s'écarte significativement du comportement appris est considéré comme une anomalie potentielle, indiquant une dégradation ou une panne imminente.

Par exemple, dans un contexte industriel, une anomalie pourrait être une augmentation inhabituelle de la température d'un capteur, signalant une usure prématurée.

Avantage majeur : Pas besoin d'exemples annotés de pannes, ce qui est idéal pour les datasets comme AI4I 2020 où les pannes sont rares (environ 3,4 %). **Limites :** Risque de faux positifs si le modèle est trop sensible (détecte des variations normales comme anomalies), ou de faux négatifs s'il est trop tolérant. De plus, il assume que les données d'entraînement sont exemptes d'anomalies cachées.

2.2 Les autoencodeurs : principe de base

Un autoencodeur est un réseau de neurones artificiels non supervisé dont l'objectif est de reproduire à la sortie l'entrée qu'il reçoit, en passant obligatoirement par une compression intermédiaire appelée espace latent ou bottleneck. Ce processus force le modèle à apprendre une représentation efficace et compacte des données.

Structure typique :

- **Encodeur** : Comprime les données d'entrée $x \in R^d$ dans un espace latent $z \in R^m$ où $m \ll d$. Mathématiquement, $z = f_\theta(x)$, avec f_θ un réseau de couches (e.g., fully connected ou LSTM).
- **Espace latent** : Représentation compressée qui capture les caractéristiques essentielles des données normales.
- **Décodeur** : Reconstitue une sortie $\hat{x} = g_\phi(z)$, où g_ϕ est le réseau symétrique.

Objectif d'entraînement : Minimiser l'erreur de reconstruction, généralement mesurée par l'erreur quadratique moyenne (MSE) :

$$\mathcal{L}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2 \quad (1)$$

où x est le vecteur d'entrée et $\hat{x}$ la reconstruction. L'optimisation se fait via backpropagation avec un optimiseur comme Adam.

Détection d'anomalies avec un autoencodeur : Les données normales sont bien reconstruites (faible MSE). Les données anormales, n'ayant pas été vues pendant l'entraînement, sont mal reconstruites (MSE élevée). On fixe un seuil τ (souvent $\mu + 3\sigma$ sur les erreurs des données de validation normales) : si $\mathcal{L}(x, \hat{x}) > \tau$, c'est une anomalie.

Dans la maintenance prédictive, cela permet de détecter des déviations subtiles dans les signaux de capteurs, comme une variation anormale de couple ou de vitesse de rotation.

Avantages : Capacité à capturer des relations non linéaires complexes ; scalable à de grandes dimensions. **Limites** : Nécessite beaucoup de données normales ; risque de sur-apprentissage si la compression est trop faible.

Autoencodeur Dense (Feedforward Autoencoder)

L'**Autoencodeur Dense**, également appelé **Feedforward Autoencoder**, est une architecture de réseau de neurones entièrement connectée utilisée pour l'**apprentissage non supervisé** des représentations de données. Son objectif est de reproduire les données d'entrée en sortie, après les avoir compressées dans un espace latent de dimension réduite. Cette compression permet d'extraire les caractéristiques les plus importantes du jeu de données, facilitant ainsi la détection d'anomalies.

Principe de fonctionnement : L'autoencodeur dense est composé de deux parties principales :

- **L'encodeur** réduit la dimension des données d'entrée en apprenant une représentation compacte et informative.
- **Le décodeur** tente de reconstruire les données initiales à partir de cette représentation compressée.

L'entraînement du modèle consiste à **minimiser l'erreur de reconstruction**, généralement mesurée par l'**erreur quadratique moyenne (MSE)**, entre les données d'entrée et celles reconstruites. Une erreur de reconstruction élevée indique qu'un échantillon diffère significativement du comportement appris comme "normal", et peut donc être considéré comme une **anomalie**.

Architecture :

- Plusieurs couches **entièrement connectées (Dense)** disposées de manière symétrique entre l'entrée et la sortie ;
- Une **couche centrale (bottleneck)** représentant l'**espace latent**, où les informations essentielles sont codées ;
- Fonctions d'activation typiques : **ReLU** pour l'encodeur et **sigmoïde** ou **linéaire** pour la couche de sortie ;
- Optimisation effectuée avec l'algorithme **Adam**.

Avantages et limites :

- Permet d'apprendre une **représentation compacte et non linéaire** des données ;
- Détecte les **valeurs aberrantes** via l'analyse de l'erreur de reconstruction ;
- Sert de **modèle de base** pour évaluer les performances d'architectures plus complexes, comme les autoencodeurs LSTM ;
- Ne prend cependant pas en compte la **dimension temporelle** des signaux : chaque observation est analysée indépendamment.

En résumé : L'Autoencodeur Dense constitue une approche simple mais efficace pour modéliser la structure normale des données et détecter les anomalies à partir de l'erreur de reconstruction. Bien qu'il soit limité pour les données temporelles, il représente une **étape de référence essentielle** avant l'utilisation de modèles récurrents capables de capturer les dépendances temporelles, tels que les

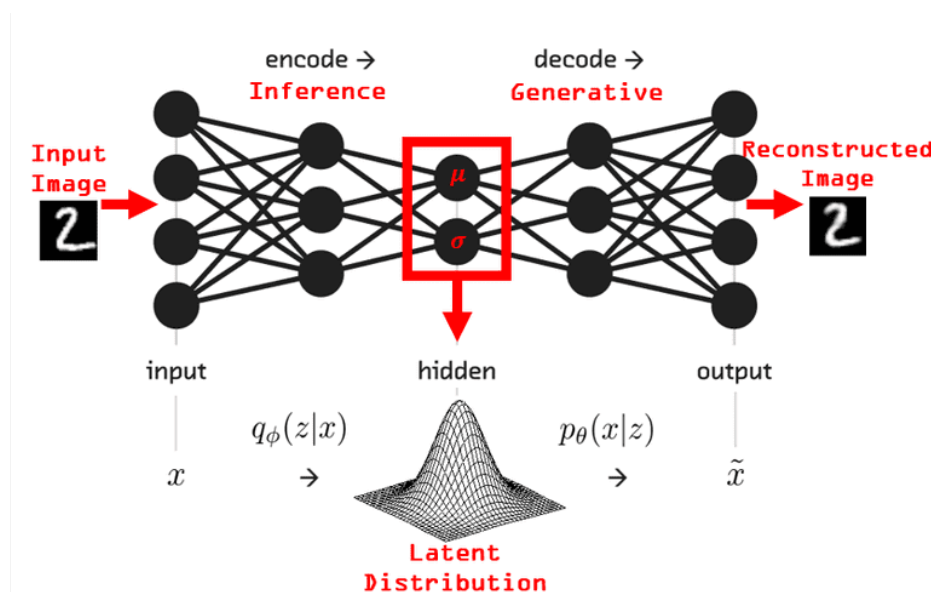


FIGURE 1 – Encodeur Dense

Autoencodeurs LSTM.

Autoencodeur LSTM

L'Autoencodeur LSTM (Long Short-Term Memory Autoencoder) est une architecture d'autoencodeur adaptée aux **séries temporelles**, reposant sur les réseaux de neurones récurrents. Il permet de capturer les **dépendances temporelles** entre observations successives et d'identifier les comportements anormaux à travers l'évolution dans le temps des signaux capteurs.

Principe de fonctionnement : L'autoencodeur LSTM se compose de deux modules :

- **L'encodeur** lit la séquence d'entrée et la compresse dans un espace latent de dimension réduite.
- **Le décodeur** reconstruit la séquence initiale à partir de cette représentation compacte.

L'objectif d'entraînement est de **minimiser l'erreur de reconstruction**, c'est-à-dire la différence entre la séquence originale et la séquence reconstruite. Une erreur élevée indique une séquence présentant un comportement anormal.

Architecture :

- Entrées sous forme de **fenêtres temporelles glissantes** de longueur fixe ;
- Deux couches LSTM pour l'encodage et le décodage ;
- Fonction de perte : **erreur quadratique moyenne (MSE)** ;
- Optimiseur : **Adam**.

Intérêt : Grâce à sa mémoire interne, le LSTM capture les relations temporelles à long terme, ce qui permet :

- De détecter les **déviations progressives** dans les signaux ;
- De distinguer les fluctuations normales des anomalies réelles ;
- De **réduire les faux positifs** par rapport à un autoencodeur dense.

Seuil de détection : Un seuil est défini à partir de la distribution des erreurs de reconstruction : toute séquence présentant une erreur supérieure à ce seuil est considérée comme une anomalie.

En résumé : L'Autoencodeur LSTM apprend la structure temporelle du comportement normal d'un système et détecte les déviations significatives, constituant une approche efficace pour la **détection d'anomalies dans les données séquentielles**.

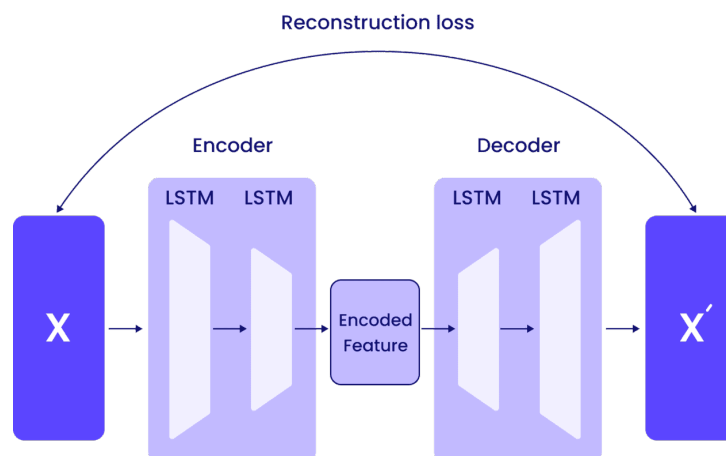


FIGURE 2 – LSTM ENCODER

2.3 Méthodes classiques de référence

Ces algorithmes sont utilisés comme baseline car ils sont plus simples, plus rapides et souvent très performants, sans nécessiter d'apprentissage profond.

Isolation Forest

L'**Isolation Forest** est une méthode d'apprentissage non supervisée conçue spécifiquement pour la **détection d'anomalies**. Elle repose sur une idée simple mais efficace : les anomalies sont des observations rares et isolées dans l'espace des données, et peuvent donc être séparées plus facilement que les points normaux.

Principe de fonctionnement : Contrairement aux algorithmes traditionnels qui modélisent le comportement normal, l'Isolation Forest procède de manière inverse : elle isole directement les anomalies en construisant de nombreux *arbres de décision aléatoires* appelés *iTrees* (Isolation Trees).

Le processus peut être résumé ainsi :

- Sélection aléatoire d'une **caractéristique** du jeu de données.
- Choix d'une **valeur de coupure aléatoire** entre le minimum et le maximum de cette caractéristique.
- Répétition récursive du processus pour partitionner les données.

Une observation anormale est généralement isolée en **peu d'étapes**, car elle se situe à la périphérie du nuage de points. À l'inverse, une observation normale nécessite davantage de partitions avant d'être isolée.

Mesure d'anomalie : L'algorithme calcule pour chaque observation la **longueur moyenne du chemin** (nombre de divisions nécessaires pour isoler un point) dans l'ensemble des arbres de la forêt.

- Une longueur de chemin **courte** indique une observation **anormale**.
- Une longueur de chemin **longue** indique une observation **normale**.

Le score d'anomalie est normalisé entre 0 et 1 :

- Score proche de **1** : forte probabilité d'anomalie.
- Score proche de **0.5** : observation normale.

Avantages :

- Méthode **rapide et scalable**, adaptée aux grands jeux de données.
- Peu de paramètres à ajuster.
- Ne fait **aucune hypothèse sur la distribution** des données.
- **Robuste** face aux valeurs aberrantes.

Limites :

- Moins performant lorsque les anomalies dépendent de relations complexes entre variables.
- Sensible à un fort déséquilibre entre classes.

En résumé : L'Isolation Forest identifie les anomalies en les **isolant rapidement** dans une forêt de partitions aléatoires, plutôt qu'en cherchant à modéliser la normalité. C'est une méthode **simple, efficace et largement utilisée** pour la détection d'anomalies en maintenance prédictive.

One-Class SVM

Le **One-Class Support Vector Machine (One-Class SVM)** est une méthode d'apprentissage non supervisée dérivée des **machines à vecteurs de support (SVM)**. Elle vise à déterminer une **frontière** qui sépare le comportement normal des observations anormales, sans nécessiter de labels préalables.

Principe de base : Le One-Class SVM cherche à identifier une région de **forte densité** représentant le comportement normal des données et considère comme anomalies les points situés en dehors de cette région.

Pour ce faire, les données d'entrée x_i sont projetées dans un **espace de dimension supérieure** à l'aide d'une fonction noyau (*kernel*), souvent le **noyau gaussien RBF (Radial Basis Function)**. Dans cet espace transformé, l'algorithme construit un **hyperplan** séparant les points normaux de l'origine tout en maximisant la marge.

Ainsi :

- Les points situés **près de l'hyperplan** sont considérés comme **normaux**.
- Les points situés **loin de l'hyperplan** sont identifiés comme **anormaux**.

Fonction d'optimisation : Le modèle apprend une fonction $f(x)$ telle que :

$$f(x) = \begin{cases} > 0 & \text{si } x \text{ est normal} \\ < 0 & \text{si } x \text{ est anormal} \end{cases}$$

Deux hyperparamètres principaux influencent son comportement :

- ν (*nu*) : proportion attendue d'anomalies dans les données (généralement entre 0.01 et 0.1).
- γ (*gamma*) : paramètre du noyau RBF qui contrôle la largeur de la région considérée comme normale.

Avantages :

- Méthode **robuste** pour des données de faible dimension.
- Capable de détecter des **frontières non linéaires** grâce à l'utilisation de noyaux.
- Convient aux contextes où **peu ou aucune étiquette** n'est disponible.

Limites :

- Sensible au choix des paramètres ν et γ .
- Moins performant sur des jeux de données **massifs ou bruités**.
- Difficulté d'interprétation dans des espaces de grande dimension.

En résumé : Le One-Class SVM apprend la forme du nuage de points représentant le comportement normal, puis identifie comme anomalies les observations qui s'en écartent. C'est une approche **mathématiquement rigoureuse** et **souple**, mais qui requiert un ajustement soigneux des hyperparamètres pour des résultats optimaux en contexte industriel.

Local Outlier Factor (LOF)

Le **Local Outlier Factor (LOF)** est une méthode non supervisée de détection d'anomalies basée sur la notion de **densité locale**. Contrairement aux approches globales comme l'Isolation Forest, le LOF ne cherche pas à définir une frontière unique entre les points

normaux et anormaux. Il évalue plutôt le **degré d'isolement local** d'une observation en comparant sa densité à celle de ses voisins immédiats.

Principe de base : Un point est considéré comme **anormal** s'il est significativement plus isolé (c'est-à-dire s'il possède une densité locale beaucoup plus faible) que ses voisins. Le calcul du LOF repose sur trois étapes principales :

1. Calcul des **k plus proches voisins (k -NN)** : identification des voisins les plus proches de chaque point selon une distance (souvent euclidienne).
2. Estimation de la **densité locale atteignable** (reachability density) : mesure de la compacité du voisinage autour du point.
3. Calcul du **facteur d'anomalie local (LOF)** : rapport entre la densité moyenne des voisins et la densité du point étudié.

Le score LOF d'un point p est défini comme :

$$LOF(p) = \frac{\text{densité moyenne des voisins de } p}{\text{densité de } p}$$

Ainsi :

- Si $LOF(p) \approx 1$, la densité de p est comparable à celle de ses voisins $\Rightarrow$ **point normal**.
- Si $LOF(p) > 1$, la densité de p est beaucoup plus faible $\Rightarrow$ **point potentiellement anormal**.

Interprétation : Le score LOF fournit une mesure continue de l'anomalie : plus le score est élevé, plus le point est considéré comme atypique. Cela permet non seulement de détecter les anomalies, mais aussi de **classer leur degré d'anormalité**.

Avantages :

- Capable de détecter des **anomalies locales** qui seraient invisibles avec une approche globale.
- Méthode **intuitive et interprétable**, ne nécessitant pas d'entraînement complexe.
- S'adapte bien aux données présentant des **régions de densité variable**.

Limites :

- Sensible au choix du paramètre k , qui influence la détection.
- Moins efficace sur les jeux de données **massifs ou bruités**.
- La complexité du calcul des distances augmente avec la **dimensionnalité**.

En résumé : Le LOF détecte les anomalies en mesurant le **contraste de densité** entre un point et son voisinage. Il constitue une méthode particulièrement utile pour identifier des anomalies locales dans des jeux de données hétérogènes, faisant de lui un outil pertinent pour la maintenance prédictive et la surveillance d'équipements industriels.

2.4 Clustering non supervisé dans l'espace latent

Une fois les données compressées par l'encodeur (espace latent), on applique du clustering pour découvrir des régimes de fonctionnement (normal, dégradé, en panne). Cela ajoute une couche d'interprétation aux autoencodeurs.

2.4.1 K-Means

Principe : On attribue chaque point à l'un des K centres (centroïdes) de manière à minimiser la somme des distances au carré (variance intra-cluster). Itératif : assignation → mise à jour des centroïdes → répétition jusqu'à convergence.

Mécanisme interne : Initialisation aléatoire ou k-means++, puis minimisation de l'inertie : $\sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$.

Adaptation : Identifie des clusters comme « normal » (majoritaire) vs « dégradé » dans l'espace latent.

Avantages : Simple, rapide. **Limites** : Assume des clusters sphériques ; besoin de fixer K (via méthode du coude ou silhouette score).

2.4.2 DBSCAN

Principe : Algorithme basé sur la densité. Un cluster est une zone dense de points (core points avec au moins MinPts voisins dans un rayon ϵ) ; les points isolés sont du bruit (outliers). Pas besoin de fixer le nombre de clusters.

Mécanisme interne : Expansion des clusters à partir de core points, incluant border points.

Adaptation : Bon pour détecter des anomalies comme bruit dans l'espace latent.

Avantages : Gère des formes arbitraires, détecte naturellement les outliers.

Limites : Sensible à ϵ et MinPts ; assume densité uniforme.

2.5 Visualisation de l'espace latent

Pour interpréter les résultats, on réduit la dimensionnalité de l'espace latent (e.g., 8D à 2D) :

- **PCA (Principal Component Analysis)** : Projection linéaire qui maximise la variance conservée. Mathématiquement, décomposition en valeurs propres de la matrice de covariance.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding)** : Projection non linéaire qui préserve les similarités locales via des probabilités (KL-divergence minimisée).

Ces méthodes permettent de visualiser si les anomalies et clusters sont bien séparés, aidant à l'interprétation en maintenance prédictive.

3 Exploration et préparation des données (EDA)

Le jeu de données AI4I 2020 contient 10 000 instances avec les colonnes suivantes : Air temperature [K] , Process temperature [K], Rotational speed [rpm], Torque [Nm] , Tool wear [min], Type (L/M/H), et les labels de panne (Machine failure binaire + types TWF, HDF, PWF, OSF, RNF).

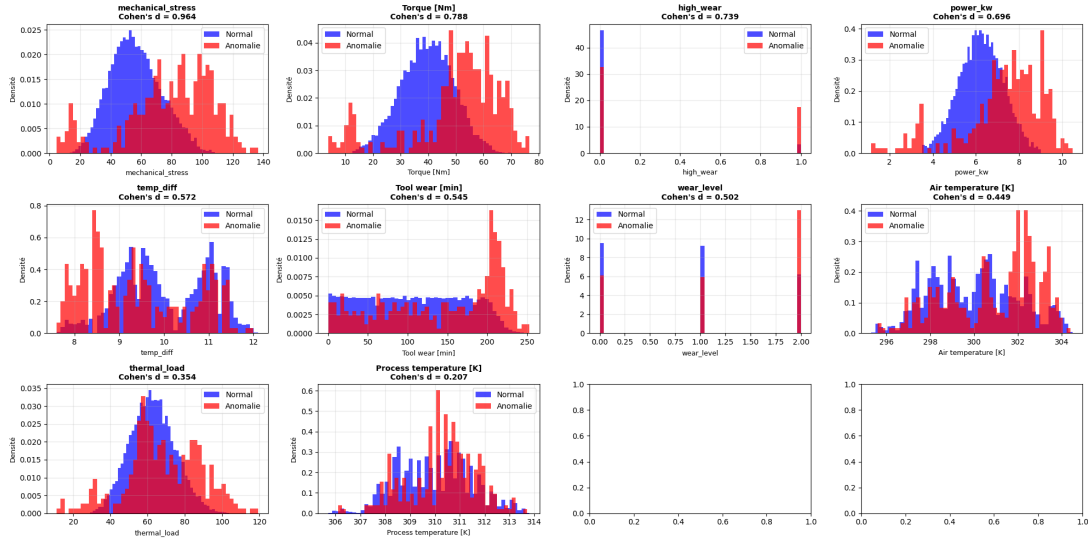


FIGURE 3 – Analyse Exploratoire

Le choix des variables pertinentes a été effectué sur la base des observations issues de l'analyse exploratoire des données.

4 Modélisation avec Autoencodeurs

4.1 Implémentation Autoencodeur Dense

Le modèle présenté met en œuvre un **Autoencodeur Dense symétrique**, conçu pour apprendre une représentation compacte des données et détecter les anomalies à partir de l'erreur de reconstruction. L'architecture se compose de deux parties : un **encodeur** et un **décodeur**.

L'**encodeur** comprend plusieurs couches denses (**Dense**) avec la fonction d'activation **ReLU**, permettant de réduire progressivement la dimension des données d'entrée. Chaque couche est associée à une **régularisation L1/L2** et à une couche de **Dropout** afin d'éviter le surapprentissage et de favoriser la généralisation du modèle. La **couche latente** (**espace_latent**) constitue le point central de l'architecture : elle encode les caractéristiques essentielles des données dans un espace de dimension réduite.

Le **décodeur** reconstruit ensuite les données originales à partir de cette représentation latente, en suivant une structure symétrique à celle de l'encodeur. Les couches de **BatchNormalization** permettent de stabiliser l'apprentissage et d'améliorer la convergence. La couche de sortie utilise une activation **linéaire**, car la tâche consiste à reproduire les valeurs d'entrée.

L'optimisation du modèle est réalisée à l'aide de l'algorithme **Adam** avec un taux d'apprentissage fixé à 0.001, et la fonction de perte choisie est l'**erreur quadratique moyenne (MSE)**, adaptée à la tâche de reconstruction. Des **callbacks** tels que **EarlyStopping** et **ReduceLROnPlateau** sont intégrés pour éviter le surapprentissage et ajuster dynamiquement le taux d'apprentissage en cas de stagnation de la validation.

Cette architecture offre un bon compromis entre **simplicité et efficacité**, et permet au modèle d'apprendre la structure sous-jacente des données normales, faisant de l'erreur de reconstruction un indicateur pertinent pour la **détection d'anomalies**.

```

# ENCODER : Comprime Les 6 features en représentation Latente (2 dimensions)
# - Couche 1 : 6 -> 8 neurones (ReLU) + Dropout(30%)
# - Couche 2 : 8 -> 4 neurones (ReLU) + Dropout(30%)
# - Couche Latente : 4 -> 2 neurones (ReLU) - goulot d'étranglement

# DECODER : Reconstitue Les 6 features originales à partir de La représentation Latente
# - Couche 1 : 2 -> 4 neurones (ReLU)
# - Couche 2 : 4 -> 8 neurones (ReLU) avec BatchNormalization
# - Couche 3 : 8 -> 8 neurones (ReLU) avec BatchNormalization
# - Output : 8 -> 6 neurones (Linear) - reconstruction sans activation

# Callbacks :
# - EarlyStopping : Arrête si validation_loss ne s'améliore pas pendant 30 epochs
# - ReduceLROnPlateau : Réduit Le Learning rate si la val_loss stagne

input_layer = keras.layers.Input(shape=(input_dim_opt,))

# ENCODER
encoded = keras.layers.Dense(8, activation='relu',
                             kernel_regularizer=regularizers.l2(l2_reg))(input_layer)
encoded = Dropout(dropout_rate, seed=SEED)(encoded)

encoded = keras.layers.Dense(4, activation='relu',
                             kernel_regularizer=regularizers.l2(l2_reg))(encoded)
encoded = Dropout(dropout_rate, seed=SEED)(encoded)

# Couche Latente
encoded = keras.layers.Dense(encoding_dim, activation='relu',
                              activity_regularizer=regularizers.l1(l1_reg),
                              kernel_regularizer=regularizers.l2(l2_reg), name='espace_latent')(encoded)

# DECODER
decoded = keras.layers.Dense(4, activation='relu',
                             )(encoded)

decoded = keras.layers.BatchNormalization()(decoded)

decoded = keras.layers.Dense(8, activation='relu',
                             )(decoded)

decoded = keras.layers.BatchNormalization()(decoded)

decoded = keras.layers.Dense(input_dim_opt, activation='linear')(decoded)

# MODÈLE COMPLET
autoencoder_opt = keras.Model(input_layer, decoded)
autoencoder_opt.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss='mse')

print("\n" + "="*60)
print("ENTRAÎNEMENT")
print("="*60)

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=30, restore_best_weights=True, verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=1e-6, verbose=1)

# Entraîner
history_opt = autoencoder_opt.fit(
    X_train_opt, X_train_opt,
    epochs=200,
    batch_size=32,
    validation_data=(X_val_opt, X_val_opt),
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

print(f"\n Entraînement terminé !")
print(f" Meilleure val_loss : {min(history_opt.history['val_loss']):.6f}")

```

FIGURE 4 – Encodeur Dense

4.2 Implémentation de l' Autoencodeur LSTM

Le modèle d'**Autoencodeur LSTM** a été implémenté à l'aide de la bibliothèque **Keras** avec le backend **TensorFlow**. L'architecture comprend une partie encodeur et une partie décodeur, chacune constituée de couches **LSTM** empilées. L'encodeur lit les séquences temporelles issues des capteurs et les compresse dans un espace latent de faible dimension, tandis que le décodeur reconstruit la séquence initiale à partir de cette représentation compacte.

La fonction de perte utilisée est l'**erreur quadratique moyenne (MSE)**, qui mesure la différence entre la séquence d'entrée et celle reconstruite. L'optimisation du modèle est assurée par l'algorithme **Adam**, choisi pour sa rapidité de convergence et sa stabilité. Après l'entraînement, les séquences présentant une erreur de reconstruction supérieure à un seuil déterminé sont considérées comme des anomalies.

Le code suivant illustre la construction et la compilation du modèle d'autoencodeur LSTM :

```

# LSTM AUTOENCODER - ARCHITECTURE BIDIRECTIONNELLE

# Construit un autoencodeur séquentiel avec LSTM pour capturer
# Les dépendances temporelles dans les séries temporelles.
#
# Architecture :
# ENCODER : Comprime la séquence en vecteur latent
# - LSTM(64) : Forward avec sorties (return_sequences=True)
# - Dropout(0.2)
# - LSTM(32) : Forward avec sortie unique par séquence (return_sequences=False)
#
# LATENT SPACE : Dense(16) - représentation compressée
#
# DECODER : Reconstitue la séquence originale
# - RepeatVector : répète le vecteur latent pour chaque timestep
# - LSTM(32) : Retour en arrière
# - Dropout(0.2)
# - LSTM(64) : Reconstruction
# - TimeDistributed(Dense) : applique Dense à chaque timestep
#
# Training : Les prédictions décodeuses seront comparées aux inputs originaux

from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input, LSTM, RepeatVector,
    TimeDistributed, Dense, Dropout
)

timesteps = X_seq.shape[1] # ex: 20
features = X_seq.shape[2] # nb de variables

inputs = Input(shape=(timesteps, features))

# ENCODER
x = LSTM(64, return_sequences=True)(inputs)
x = Dropout(0.2)(x)

x = LSTM(32, return_sequences=False)(x)

# Latent space
latent = Dense(16, activation="relu")(x)

# DECODER
x = RepeatVector(timesteps)(latent)

x = LSTM(32, return_sequences=True)(x)
x = Dropout(0.2)(x)

x = LSTM(64, return_sequences=True)(x)

outputs = TimeDistributed(Dense(features))(x)

# MODEL
autoencoder = Model(inputs, outputs)
autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

autoencoder.summary()

```

FIGURE 5 – Encodeur LSTM

5 Comparaison avec les méthodes classiques

Après la mise en œuvre des différentes approches de détection d'anomalies, une phase de **comparaison des performances** a été réalisée afin d'évaluer l'efficacité de chaque

méthode. Cette étape vise à identifier le modèle le plus performant pour la détection des comportements anormaux dans le cadre de la maintenance prédictive.

Les modèles considérés incluent les approches classiques (**Isolation Forest**, **One-Class SVM**, **Local Outlier Factor**) et les approches neuronales basées sur des **Autoencodeurs** (Dense et LSTM). Chaque modèle a été évalué selon des indicateurs de performance standards tels que la **précision**, le **rappel**, le **F1-score**, ainsi que la **matrice de confusion**.

L'analyse de ces résultats permet de comprendre la capacité de chaque méthode à distinguer les observations normales des anomalies, ainsi que leurs limites respectives. Les sections suivantes présentent, pour chaque modèle, la matrice de confusion obtenue et les scores obtenues pour chaque cas.

Cas 1 : Methode Isolation Forest

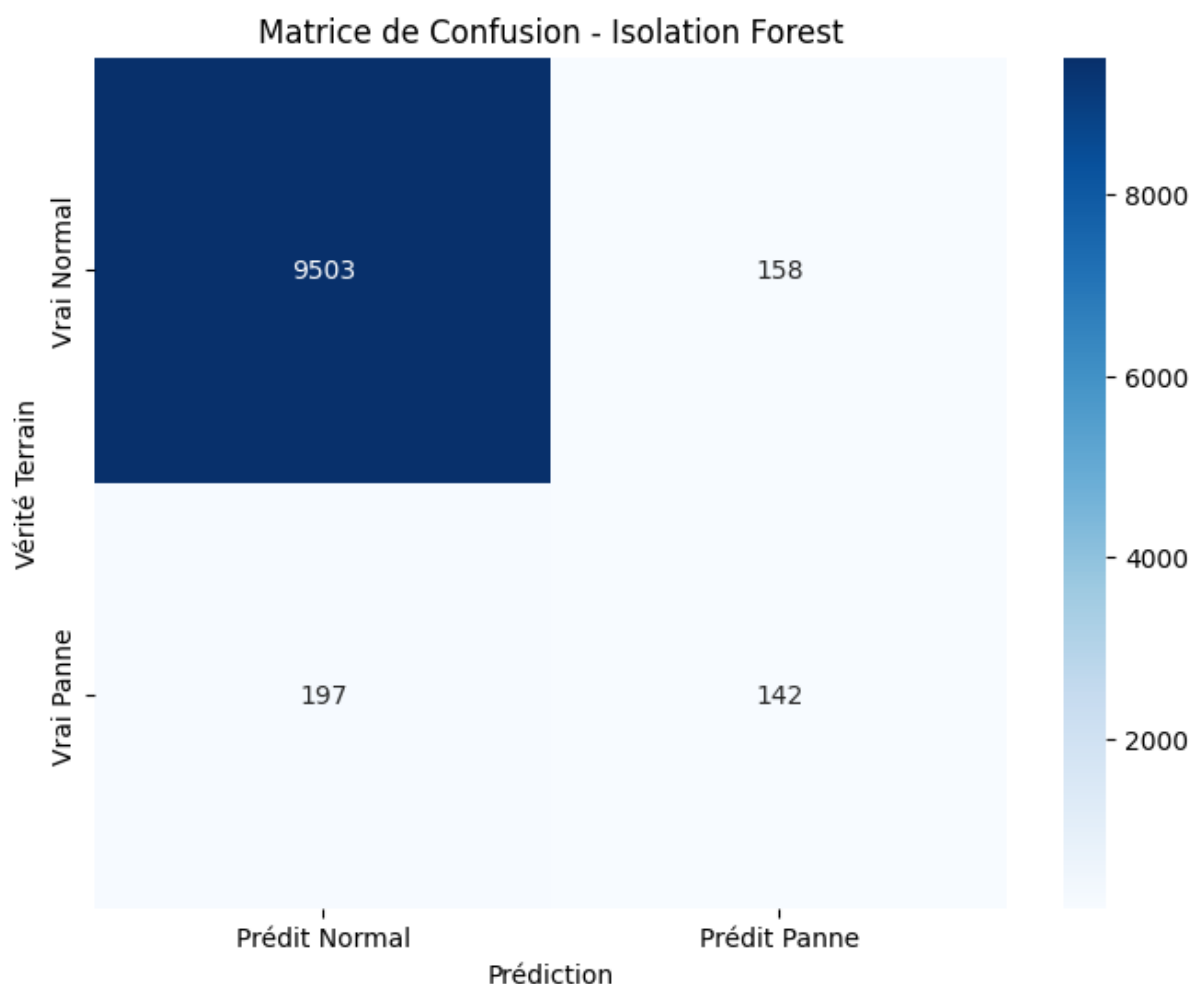


FIGURE 6 – Matrice de confusion isolation forest

```

=====
RÉSULTATS ISOLATION FOREST
=====

Matrice de Confusion :
[[9503  158]
 [ 197  142]]

- Pannes détectées (TP) : 142
- Pannes ratées (FN)   : 197
- Fausses alarmes (FP) : 158

```

FIGURE 7 – Score Isolation Forest

Cas 2 : Méthode One-Class SVM

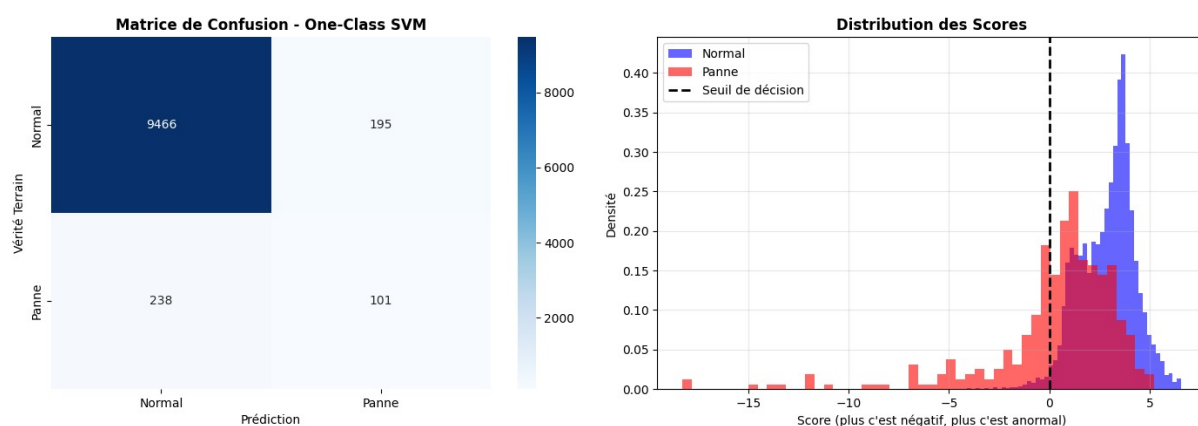


FIGURE 8 – Méthode One-Class SVM

	precision	recall	f1-score	support
Normal	0.98	0.98	0.98	9661
Panne	0.34	0.30	0.32	339
accuracy			0.96	10000
macro avg	0.66	0.64	0.65	10000
weighted avg	0.95	0.96	0.96	10000

FIGURE 9 – Score One Class SVM

Cas 3 : Méthode LOF

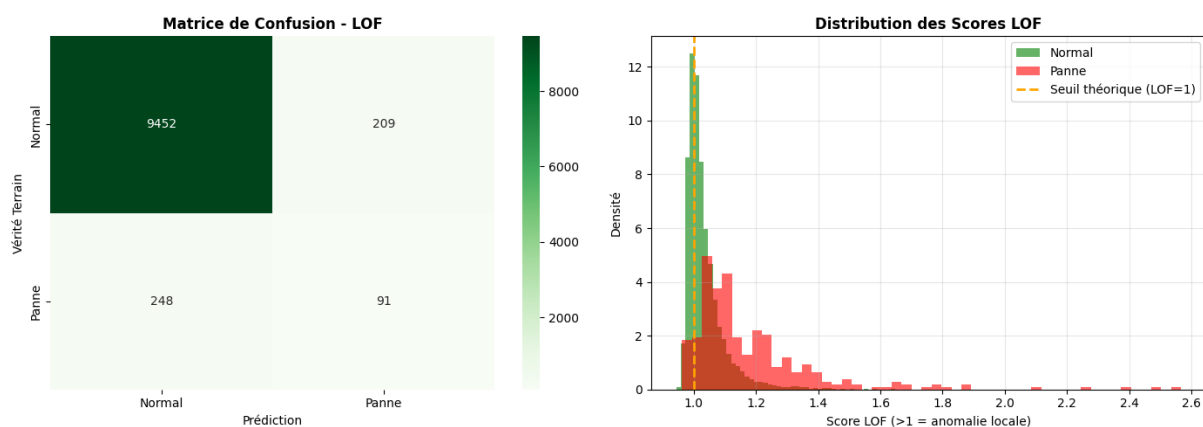


FIGURE 10 – Matrice de confusion LOF

	precision	recall	f1-score	support
Normal	0.97	0.98	0.98	9661
Panne	0.30	0.27	0.28	339
accuracy			0.95	10000
macro avg	0.64	0.62	0.63	10000
weighted avg	0.95	0.95	0.95	10000

FIGURE 11 – Score LOF

Cas 4 : Méthode LSTM

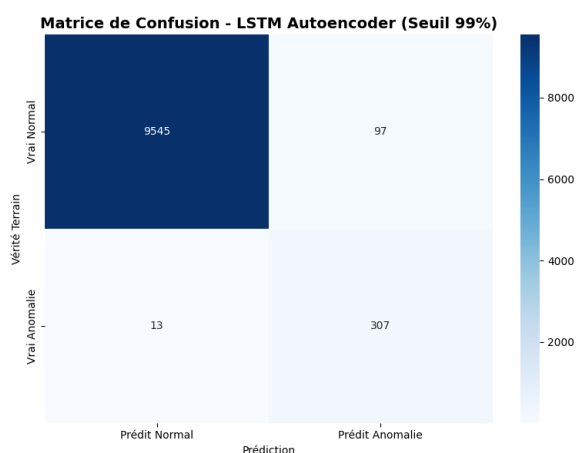


FIGURE 12 – Matrice de confusion LSTM

	precision	recall	f1-score	support
Normal	1.00	0.99	0.99	9642
Anomalie	0.76	0.96	0.85	320
accuracy			0.99	9962
macro avg	0.88	0.97	0.92	9962
weighted avg	0.99	0.99	0.99	9962

FIGURE 13 – Score LSTM

5.0.1 Interprétation des résultats des modèles de détection de pannes

Les résultats comparent les performances de quatre modèles sur un jeu de données fortement déséquilibré (environ 97 % d'états normaux et 3 % de pannes).

L'Isolation Forest présente une détection limitée des pannes, avec un recall d'environ 42 % (142 vraies positives sur 339), 197 pannes manquées et 158 fausses alarmes. Le modèle privilégie la précision sur la classe majoritaire au détriment de la détection des anomalies rares.

Le One-Class SVM obtient d'excellents scores sur la classe « Normal » (F1-score 0,98), mais des performances très faibles sur la classe « Panne » (precision 0,34, recall 0,30, F1-score 0,32). Il rate ainsi 70 % des pannes réelles, ce qui le rend inadapté à un usage opérationnel.

Le Local Outlier Factor (LOF) affiche des résultats encore plus médiocres sur la classe minoritaire (recall 0,27, F1-score 0,28), confirmant son inefficacité parmi les approches non supervisées testées.

En revanche, le modèle LSTM atteint les meilleures performances globales (accuracy 0,99) et surtout sur la classe « Anomalie » (precision 0,76, recall 0,96, F1-score 0,85). Il détecte 96 % des pannes tout en limitant raisonnablement les fausses alarmes, offrant un équilibre nettement supérieur.

Conclusion : Les méthodes non supervisées classiques (Isolation Forest, One-Class SVM, LOF) s'avèrent inefficaces pour détecter les pannes dans un contexte de fort déséquilibre de classes, car elles ratent trop d'anomalies critiques. Le modèle LSTM, qui exploite la dimension temporelle des données, fournit une détection fiable et opérationnelle, et constitue clairement la solution la plus performante pour une application de maintenance prédictive.

6 Clustering dans l'espace latent

Consiste à appliquer des algorithmes de clustering sur les vecteurs latents générés par un autoencodeur ou un modèle de représentation similaire. Ces vecteurs compressent l'infor-

mation originale tout en conservant les caractéristiques discriminantes des données. Les points proches dans cet espace indiquent des comportements ou patterns similaires, permettant une segmentation précise, la détection d'anomalies et la réduction du bruit. Cette méthode améliore la séparabilité des clusters et facilite l'analyse de données complexes dans les systèmes logiciels.

```
# EXTRACTION DE L'ESPACE LATENT (ENCODER)

# Créons un modèle 'encoder' qui prend l'entrée de l'autoencodeur et retourne la sortie de la couche nc
# Utilisé pour extraire des représentations basses-dimensionnelles qui résument le comportement du syst

from tensorflow import keras

# Modèle encodeur : entrée → espace latent
encoder = keras.Model(
    inputs=autoencoder_opt.input,
    outputs=autoencoder_opt.get_layer("espace_latent").output
)

# Affichons un résumé de l'encodeur
encoder.summary()
```

FIGURE 14 – Espace Latente

7 Visualisation et interprétation

Dans cette section, nous présentons les différentes visualisations réalisées à l'aide de Streamlit, permettant d'analyser le comportement des données, d'identifier les anomalies détectées par chaque modèle et de comparer leurs performances à travers des représentations claires et interactives. ,

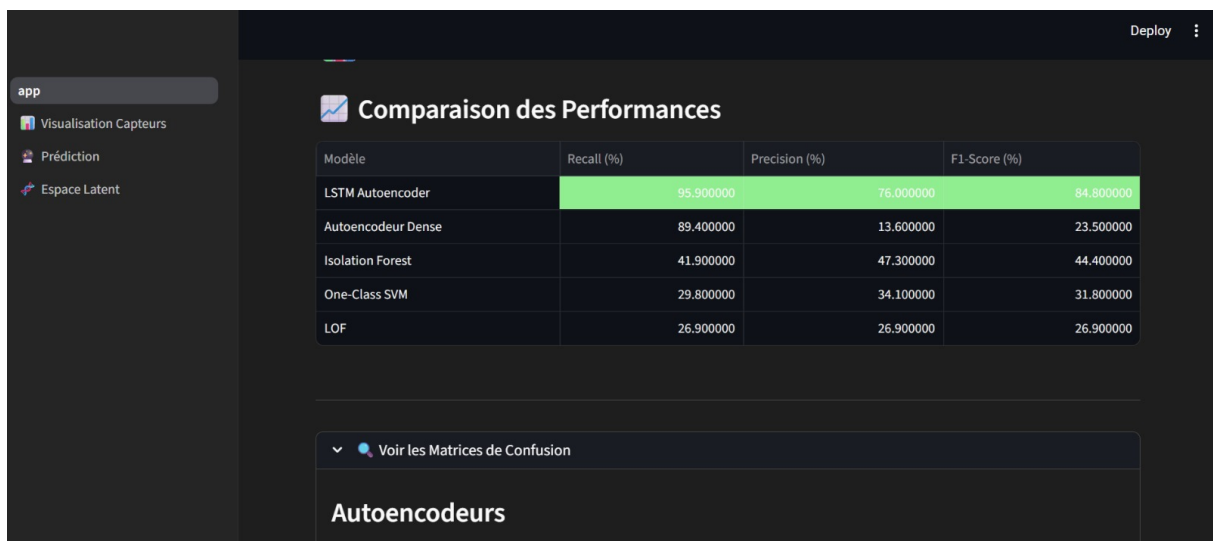


FIGURE 15 – Interpretation

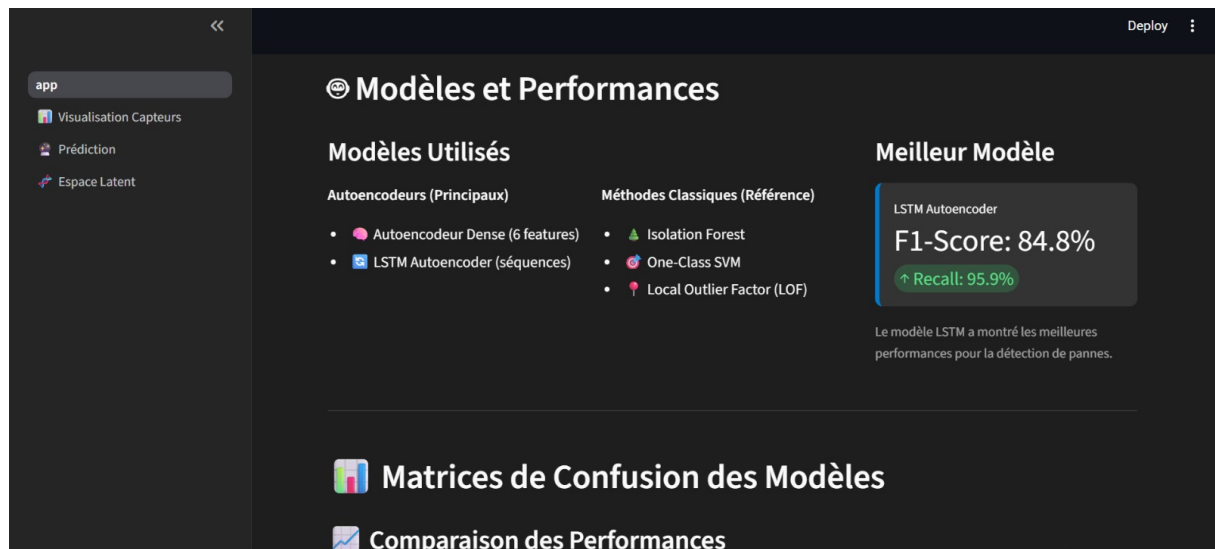


FIGURE 16 – Resultats et Intrepretations

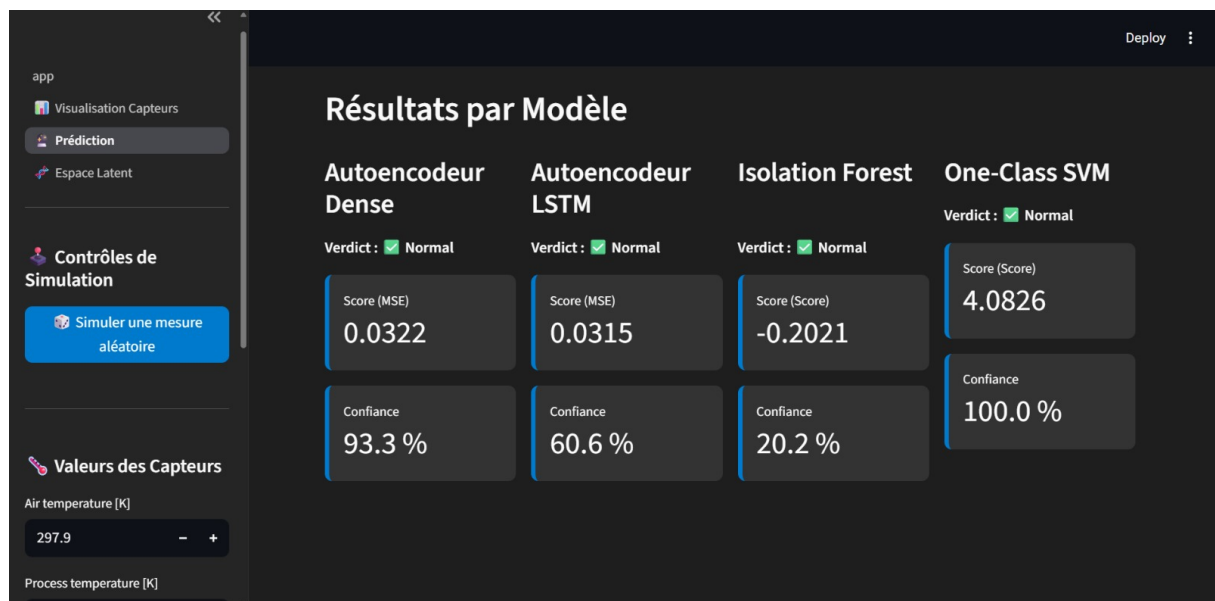


FIGURE 17 – Resultats et Intrepretations

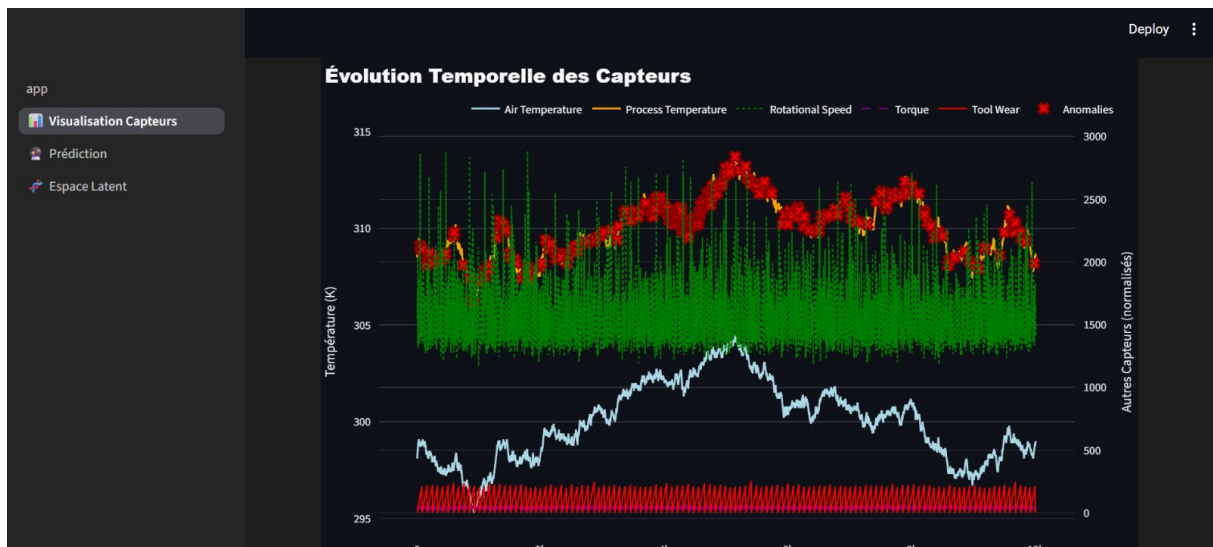


FIGURE 18 – Interpretations et visualisations



FIGURE 19 – Interpretations et visualisations

8 Développement du tableau de bord Streamlit

Framework Python léger qui transforme automatiquement des scripts en applications web interactives. Il permet d'intégrer facilement des widgets interactifs (boutons, sliders, sélecteurs), des visualisations dynamiques (graphiques, tableaux) et des mises à jour en temps réel sans écrire de HTML, CSS ou JavaScript. Idéal pour prototyper des dashboards, tester des modèles de machine learning et créer des interfaces réactives pour l'analyse et l'exploration de données dans un environnement logiciel professionnel.

Le Tableau de bord **Streamlit** affiche :

- Les courbes de capteurs
- Les anomalies détectées par chaque méthode

— Les clusters de l'espace latent
Automatisé pour temps réel.

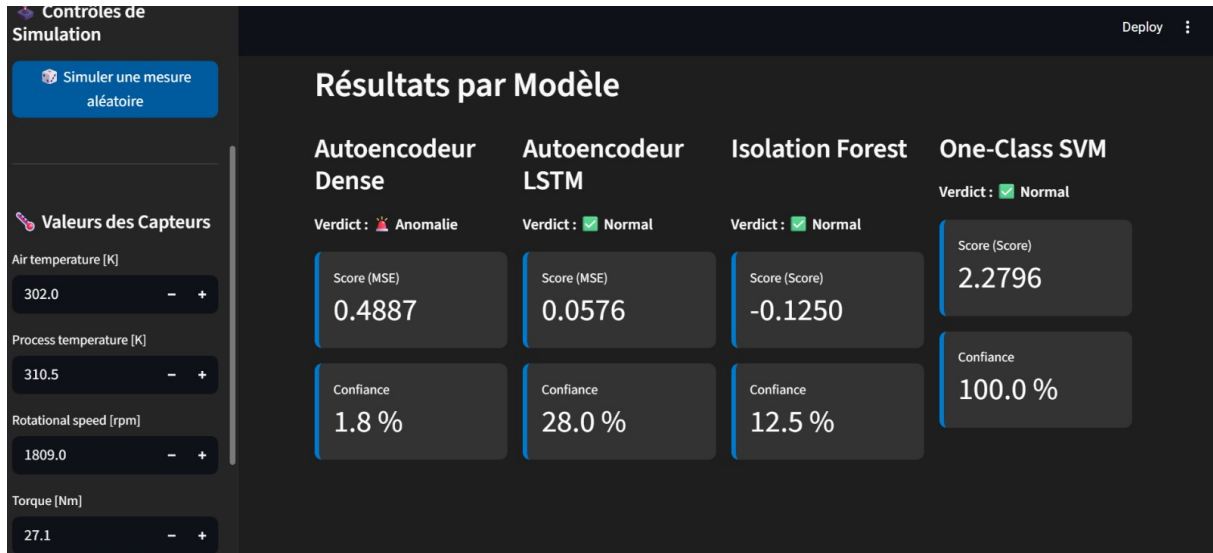


FIGURE 20 – Streamlit

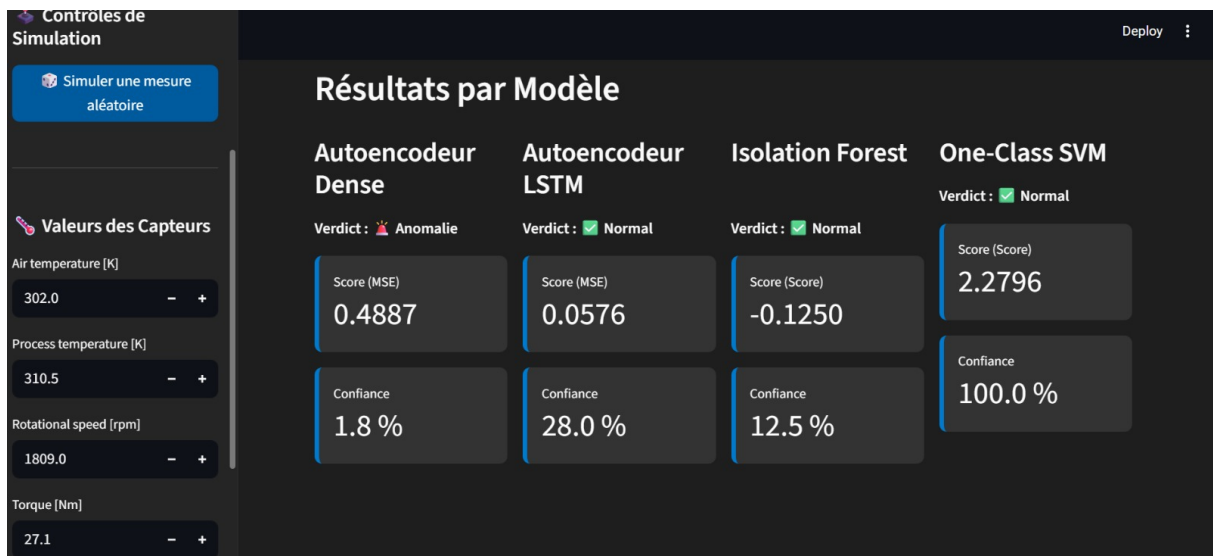


FIGURE 21 – Streamlit

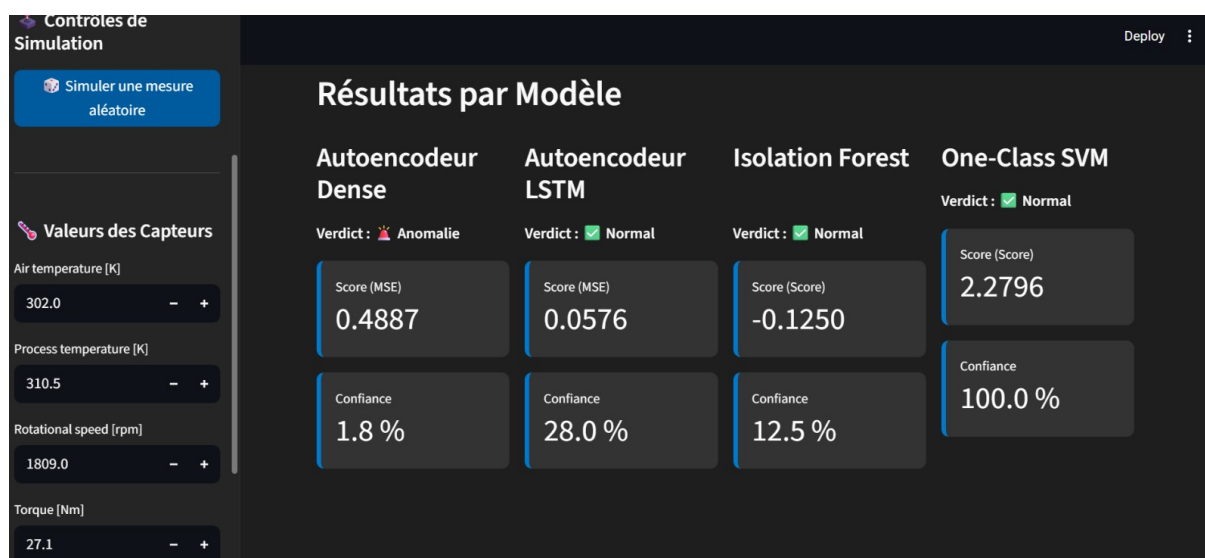


FIGURE 22 – Streamlit

9 Conclusion

Ce travail a exploré la détection d'anomalies non supervisée dans un contexte de maintenance prédictive, en s'appuyant sur des fondements théoriques solides et une analyse approfondie des données. Après une phase d'exploration et de préparation (EDA), plusieurs approches ont été mises en œuvre, allant des méthodes classiques de référence aux autoencodeurs modernes.

Les résultats montrent que les autoencodeurs, en particulier l'autoencodeur LSTM, offrent des performances supérieures pour la modélisation de comportements complexes et temporels. Leur capacité à apprendre des représentations compactes permet une détection plus fine des anomalies par rapport aux méthodes traditionnelles telles que l'Isolation Forest ou le One-Class SVM.

L'intégration du clustering dans l'espace latent a permis d'identifier différents régimes de fonctionnement et d'améliorer l'interprétation des résultats. Les techniques de visualisation associées, renforcées par le développement d'un tableau de bord Streamlit, ont facilité l'analyse, la comparaison des modèles et la prise de décision grâce à une exploration interactive des données et des anomalies détectées.

Malgré ces résultats encourageants, certaines limites subsistent, notamment l'utilisation d'un jeu de données synthétique. Les perspectives futures incluent l'exploitation de données réelles, l'intégration de modèles plus avancés tels que les autoencodeurs variationnels (VAE), ainsi que l'extension du système vers des solutions plus robustes et déployables en environnement industriel réel.